Draft version April 15, 2026
Typeset using LaTeX **preprint** style in AASTeX631

Validation of SWRF-Impute: MCAR, MAR, and MNAR Testing

Agastya Gaur

1. INTRODUCTION

Validation of an imputation algorithm is essential to assess its ability to reconstruct missing values accurately while preserving statistical properties of the dataset. The reliability of imputation depends critically on the missing data mechanism. This document presents validation methodology and completed results for the SWRF-Impute imputation algorithm under three distinct missing data mechanisms: Missing Completely At Random (MCAR), Missing At Random (MAR), and Missing Not At Random (MNAR).

2. DATA AND FEATURES

The validation studies employ exoplanet host star data from the NASA Exoplanet Archive stored in `data/cleaned/STELLARHOSTS.csv`. The available features in the full dataset are listed in Table 1.

Parameter	**Description**
`sy_name`	System Name
`hostname`	Host Name
`sy_snum`	Number of Stars
`sy_pnum`	Number of Planets
`st_refname`	Stellar Parameter Reference
`st_spectype`	Spectral Type
`st_teff`	Stellar Effective Temperature [K]
`st_rad`	Stellar Radius [Solar Radius]
`st_mass`	Stellar Mass [Solar mass]
`st_met`	Stellar Metallicity [dex]
`st_metratio`	Stellar Metallicity Ratio
`st_lum`	Stellar Luminosity [log(Solar)]
`st_logg`	Stellar Surface Gravity [log10(cm/s^2)]
`st_age`	Stellar Age [Gyr]
`st_dens`	Stellar Density [g/cm^3]
`st_vsin`	Stellar Rotational Velocity [km/s]
`st_rotp`	Stellar Rotational Period [days]
`st_radv`	Systemic Radial Velocity [km/s]

Table 1. Data parameters collected from the NASA Exoplanet Archive STELLARHOSTS Table. More information can be found at https://exoplanetarchive.ipac.caltech.edu/docs/API_STELLARHOSTS_columns.html.

2.1. *Dynamic Feature Selection*

Rather than using a fixed set of features, the analysis uses a dynamic selection mechanism. Dynamic feature selection is controlled via a completeness threshold parameter τ, which determines the maximum allowable missing fraction for a feature to be included in the analysis. The data matrix $\mathbf{X} \in \mathbb{R}^{N \times P}$ is constructed by selecting only features p where the fraction of missing values satisfies:

$$\text{missing_fraction}(p) \leq \tau \tag{1}$$

This flexible approach allows validation studies to be conducted at different completeness levels by adjusting τ, enabling investigation of algorithm robustness across varying data quality scenarios.

For this paper, we employ $\tau = 0.7$ (70% completeness threshold), which yields $P = 9$ features and $N = 583$ complete-case samples where all selected features are fully observed.

2.2. *Initializer Matrix*

The complete-case dataset is supplemented with an initializer matrix $\mathbf{X}^{init} \in \mathbb{R}^{M \times P}$ containing draws from empirical prior distributions. For each selected feature, the prior distribution is obtained from:

1. **Gaia-sourced features**: Empirical distributions from Gaia DR3 astrometric measurements
2. **Archive-only features**: Empirical distributions from observed values in the full STELLARHOSTS dataset

This dual-source initialization strategy provides informative priors based on external calibrations (Gaia) where available, while leveraging STELLARHOST statistics otherwise. The initializer provides empirical distributions for the SWRF-Impute algorithm's initialization step.

3. EVALUATION METRICS

Two metrics assess imputation quality:

3.1. *Imputation Accuracy: RMSE on Masked Entries*

The primary metric quantifies the pointwise accuracy of imputed values. Given the original complete data $\mathbf{X}_{orig}$, imputed data $\mathbf{X}_{imp}$, and binary mask $\mathbf{M}$ indicating missing locations, we compute:

$$\text{RMSE}_{\text{masked}} = \sqrt{\frac{1}{\sum_{i,j} \mathbf{M}_{i,j}} \sum_{i,j} \mathbf{M}_{i,j} (\mathbf{X}_{orig,i,j} - \mathbf{X}_{imp,i,j})^2} \tag{2}$$

This metric measures how close imputed values are to the ground truth on missing entries only, capturing the algorithm's pointwise predictive accuracy. Lower values indicate better imputation quality.

3.2. *Pattern Preservation: Eigenvalue Spectrum Distance*

The second metric assesses whether imputation preserves the statistical structure and correlations in the data. After mean-centering and standardizing both the original and imputed data, Principal Component Analysis (PCA) is performed to obtain eigenvalue spectra λ_{true} and λ_{imp} representing the variance structure.

We quantify the distance between spectra using logarithmic L^2 distance:

$$d_{\text{eig}} = \sqrt{\sum_{k=1}^{P} (\log(\lambda_{true,k} + \epsilon) - \log(\lambda_{imp,k} + \epsilon))^2} \tag{3}$$

where $\epsilon = 10^{-8}$ provides numerical stability. This log-scale distance is appropriate for eigenvalues spanning multiple orders of magnitude and emphasizes relative differences. Lower values indicate that the imputation preserves the correlational structure of the data, which is critical for downstream analysis.

4. VALIDATION METHODOLOGY

Across all three missing data mechanisms (MCAR, MAR, and MNAR), the validation pipeline is identical except for mask generation.

4.1. *Initialization*

All tests begin with the same setup. Starting from the set of complete-case samples $\mathbf{X}_{known} \in \mathbb{R}^{N \times P}$ with $\tau = 0.7$, we systematically introduce missingness:

$$n_{\text{missing}} = \lfloor \text{prop_missing} \times (N \times P) \rfloor \tag{4}$$

where prop_missing ranges from 0.1 (10%) to 0.6 (60%) in increments of 0.1. The missingness is applied via mechanism-specific mask generation (see §5 for details on MCAR, MAR, and MNAR mask creation).

4.2. *Imputation Algorithm Comparison*

The following six algorithms are evaluated identically for all three mechanisms:

1. **Mean Imputation** - Replaces missing values with the mean of observed values in each feature.

2. **Median Imputation** - Replaces missing values with the median of observed values in each feature.

3. **k-Nearest Neighbors (KNN)** - Imputes using the average of the 5 nearest complete neighbors in standardized feature space.

4. **MICE** - Multivariate Imputation by Chained Equations with maximum 10 iterations and posterior sampling to introduce stochasticity.

5. **MissForest** - Iterative imputation using Random Forest regressors (30 trees, 10 iterations), unweighted.

6. **SWRF-Impute** - The proposed method: weighted stochastic imputation via random forests (30 trees, 10 iterations) with truncated normal sampling.

4.3. *Stochastic Method Handling*

All stochastic methods (MICE, MissForest, and SWRF-Impute) are evaluated on a pooled point estimate: multiple imputed matrices are generated and then averaged cell-wise into a single matrix before computing RMSE and eigenvalue distance. Evaluating all stochastic methods on the pooled matrix, rather than averaging per-run metrics, places them on equal footing and is consistent with the Rubin's rules approach to multiple imputation.

1. **MICE**: executed $n_{\text{runs}} = 35$ independent times per missingness level, each with a distinct seed drawn from a seeded random generator. The 35 resulting imputed matrices are averaged cell-wise to form the pooled point estimate used for evaluation.

2. **MissForest**: executed $n_{\text{runs}} = 35$ independent times per missingness level, each with a distinct seed, and the 35 imputed matrices are averaged cell-wise. Unlike simple deterministic use of MissForest (single run with a fixed seed), this accounts for the variance introduced by Random Forest bootstrap sampling.

3. **SWRF-Impute**: a single Gibbs-style chain is run with $n_{\text{iters}} = 15$ iterations and draws_per_iter $= 3$, producing 45 total draws. After discarding a burn-in of $b = 10$ draws, the remaining 35 post-burn-in draws are averaged cell-wise to form the pooled point estimate.

4.4. *Reproducibility*

All mask generation and stochastic imputation is driven by a single seeded `numpy.random.Generator` (via `numpy.random.default_rng`). A master seed initialises the generator; independent child generators for each (iteration, missingness level) pair are derived by drawing 32-bit integer seeds from the master. This ensures all experiments are fully reproducible while keeping individual mask realisations statistically independent of one another.

The seed thread runs through the entire pipeline. SWRF-Impute's internal random operations—initialisation draws from the prior distribution, Random Forest bootstrap sampling, and truncated-normal draws at each Gibbs step—all accept the child generator directly. MICE and MissForest receive integer seeds derived from the same master generator for each of their n_{runs} calls.

4.5. *Result Aggregation*

A single full test consists of:

- Evaluation at 6 missingness levels: $\{0.1, 0.2, \ldots, 0.6\}$
- 6 imputation algorithms

The test can be repeated across multiple iterations (n_{tot} trials) to assess stability. Final performance metrics are averaged across all iterations, producing two result dictionaries indexed by mechanism and missingness level.

$$\text{RMSE}_{\text{results}} = \{\text{algorithm} : [\text{RMSE}_{0.1}, \ldots, \text{RMSE}_{0.9}]\} \tag{5}$$

$$\text{Eigval}_{\text{results}} = \{\text{algorithm} : [d_{\text{eig},0.1}, \ldots, d_{\text{eig},0.9}]\} \tag{6}$$

5. MASK GENERATION METHODS

The primary difference across MCAR, MAR, and MNAR testing is the mask generation strategy. All other pipeline components (initialization, imputation, aggregation) are identical.

5.1. *MCAR*

In MCAR data, missing data is completely random. To simulate this for each missingness level, a single random mask is generated for each full test. The mask is created by:

1. Flattening the complete data matrix to a vector of length $N \times P$
2. Randomly sampling n_{missing} indices without replacement
3. Converting indices back to (row, column) pairs via unravel_index()
4. Setting masked entries to NaN to create $\mathbf{X}^{\text{MCAR}}_{\text{masked}}$

5.2. *MAR*

In MAR data, the probability that a value is missing depends on observed variables. We generate MAR missingness by selecting a random feature as a trigger variable and masking features across rows conditioned on extreme values in that trigger. This creates realistic patterns where missingness depends on observable, but extreme, values. This method is loosely based on the MAR generation method used by Toye et al. (2025).

1. **Select trigger feature**: Randomly choose one feature f_{trigger} from the P available features
2. **Standardize trigger feature**: Compute z-scores for all observations in the trigger feature: $z_i = \frac{f_{\text{trigger},i} - \mu}{\sigma}$ for $i = 1, \ldots, N$
3. **Rank by extremity**: Sort observations by the absolute value of z-score $|z_i|$ in descending order (most extreme values first)
4. **Mask row features**: For the top $\lceil \text{prop_missing} \times N \rceil$ observations (ranked by extremity), mask all features except the trigger feature f_{trigger}. This ensures observations with extreme trigger values lose information across multiple features but retain the trigger itself.
5. **Fill remaining mask**: If the total number of masked cells is less than n_{missing}, randomly select from remaining unmasked cells until the target proportion is reached.

This mechanism creates MAR patterns where missingness is ignorable given the observed trigger feature. Observations with extreme trigger values (high $|z|$) experience widespread missingness in other features, but the trigger feature remains fully observed.

5.3. *MNAR*

In MNAR data, the probability that a value is missing depends on the unobserved value itself or on hidden confounders. To generate this, we employ a version of the Missing Based on Unobserved Variables (MBUV) strategy introduced by Pereira et al. (2024).

Our version of the algorithm operates as follows:

1. **Generate unobserved feature**: Generate a standardized unobserved feature $f_{\text{unobserved}}$ following a normal distribution.
2. **Rank by extremity**: Sort observations of $f_{\text{unobserved}}$ by the absolute value of z-score $|z_i|$ in descending order (most extreme values first)
3. **Mask row features**: For the top $\lceil \text{prop_missing} \times N \rceil$ observations (ranked by extremity), mask all but one randomly selected feature.
4. **Fill remaining mask**: If the total number of masked cells is less than n_{missing}, randomly select from the remaining unmasked cells to reach the target proportion.

REFERENCES

Pereira, R. C., Abreu, P. H., Rodrigues, P. P., & Figueiredo, M. A. T. 2024, Expert Systems with Applications, 249, 123654, doi: 10.1016/j.eswa.2024.123654

Toye, A. A., Celik, A., & Kleinberg, S. 2025, Proceedings of machine learning research, 287, 480